

ContentSphere

Infrastructure as Code Deployment and Automation

PROJECT REPORT

Primary Tool: Terraform

Cloud: Amazon Web Services (AWS)

Region: ap-south-1 (Mumbai)

Author: Shubham Gaonkar

A practical AWS Infrastructure as Code project covering Terraform provisioning, EC2 web deployment, remote state, state management, drift concepts, CloudFormation, AWS CDK and GitHub.

1. Project Overview

ContentSphere demonstrates how AWS infrastructure can be provisioned and managed using Infrastructure as Code. Terraform is the primary implementation. The project creates networking, security, compute, an Elastic IP and S3 storage, then hosts a ContentSphere website on an Ubuntu EC2 instance using Nginx.

2. Objectives

- Learn Terraform from beginner level.
- Provision AWS infrastructure reproducibly.
- Use variables, outputs and Terraform lifecycle commands.
- Organize infrastructure into reusable modules.
- Deploy an EC2 web server and Nginx website.
- Move Terraform state to an encrypted, versioned S3 backend.
- Understand Terraform state, refresh and drift detection.
- Explore CloudFormation and AWS CDK as alternative IaC tools.
- Maintain the project using Git and GitHub.

3. Technologies Used

Technology	Purpose
Terraform	Primary Infrastructure as Code tool
AWS EC2	Ubuntu web server
AWS VPC	Network boundary
AWS S3	Storage and Terraform remote state
Elastic IP	Stable public address
Security Group	Network access control
Nginx	Web server
CloudFormation	AWS-native IaC demonstration
AWS CDK	Programmatic IaC demonstration
Git/GitHub	Version control and repository hosting

4. AWS Architecture

The deployed architecture follows this flow:

Internet → Internet Gateway → VPC 10.0.0.0/16 → Public Subnet 10.0.1.0/24 → EC2 t3.micro → Nginx
→ ContentSphere Website

The EC2 instance uses a security group for SSH and HTTP access and an Elastic IP for a stable public endpoint.

5. Terraform Resources

Resource	Implementation
VPC	10.0.0.0/16
Public Subnet	10.0.1.0/24 in ap-south-1a
Internet Gateway	Provides internet connectivity

Route Table	Default route 0.0.0.0/0 to IGW
Security Group	SSH 22 and HTTP 80 ingress
EC2	Ubuntu t3.micro
Elastic IP	Stable public IP
S3 Bucket	ContentSphere storage

6. Terraform Project Structure

```

Contentsphere/
  ├── main.tf
  ├── provider.tf
  ├── variables.tf
  ├── outputs.tf
  ├── terraform.tfvars
  ├── modules/
  │   ├── vpc/
  │   ├── ec2/
  │   ├── security-group/
  │   ├── elastic-ip/
  │   ├── s3/
  │   ├── cloudformation/
  │   ├── cdk/
  │   ├── documentation/
  │   └── screenshots/

```

7. Terraform Lifecycle

The project practiced terraform init, plan, apply and destroy. Infrastructure was destroyed and recreated during the lab, demonstrating the declarative lifecycle. Targeted destroy was explored as a learning exercise.

8. ContentSphere Website

Nginx was installed on the EC2 Ubuntu server and configured to serve a professional ContentSphere dashboard showing infrastructure status, AWS resources, deployment flow and project information.

Terraform → AWS VPC → EC2 → Nginx → ContentSphere Website

9. Remote Terraform State

A dedicated S3 bucket named contentsphere-terraform-state-048674617574 was created for Terraform state. S3 versioning and AES256 server-side encryption were enabled. Existing local state was migrated with terraform init -migrate-state. The backend uses the key contentsphere/terraform.tfstate.

10. Terraform State Management

The state was inspected using terraform state list and terraform state show. The EC2 state showed the running instance ID, AMI, instance type, subnet, security group, private/public IPs and tags. Refresh-only state synchronization was also explored.

```

terraform state list
terraform state show aws_instance.contentsphere_server
terraform plan
terraform apply -refresh-only

```

11. Infrastructure Drift

Drift occurs when AWS infrastructure is changed outside Terraform. The project included a manual EC2 tag change to demonstrate how terraform plan can detect differences between the declared configuration and actual AWS resources.

12. CloudFormation and AWS CDK

The repository contains CloudFormation and CDK directories for comparison with Terraform. The CDK project was initialized with Python and AWS CDK 2.1144.0. The CDK synthesis step encountered a missing `aws_cdk` Python dependency, so the CDK portion should be reported as initialized but not fully deployed.

13. GitHub Repository

The project was initialized as a Git repository. Terraform state files, AWS credential material and virtual-environment files were excluded through `.gitignore`. The repository was pushed to GitHub.

https://github.com/shubhamgaonkar0797/Terraform_P5.git

14. Task Completion Summary

Area	Status
Terraform/AWS setup	Completed
AWS infrastructure	Completed
Variables and outputs	Completed
Lifecycle operations	Completed
Reusable modules	Partially completed
Module-based deployment	Partially completed
Website deployment	Completed
S3 remote state	Completed
State management	Completed
Drift detection	Practiced
CloudFormation	Included as demonstration
AWS CDK	Initialized; dependency issue remained
GitHub repository	Completed
README/report	Completed

15. Security and Best Practices

Terraform state files were excluded from Git tracking. The remote state bucket uses encryption and versioning. AWS credentials and private key files were excluded from the repository. A separate S3 bucket was used for Terraform state rather than the application storage bucket.

16. Learning Outcomes

The project provided hands-on experience with AWS networking, EC2, S3, Elastic IP, security groups, Terraform configuration, modules, remote state, state inspection, lifecycle operations, drift concepts, Linux administration, Nginx, CloudFormation, AWS CDK and GitHub.

17. Conclusion

ContentSphere demonstrates a practical AWS Infrastructure as Code workflow. The strongest completed deliverable is the Terraform-managed AWS environment with a working Nginx website, remote S3 Terraform state and GitHub version control. The repository also provides a foundation for completing the remaining module migration and CDK synthesis/deployment work.